

UNIVERSITY OF
KWAZULU-NATAL

INYUVESI
YAKWAZULU-NATALI

SCHOOL OF ELECTRICAL, ELECTRONIC AND COMPUTER ENGINEERING

ENEL3SF –SOFTWARE ENGINEERING 2

GROUP 10

NAMES	STUDENT NUMBER
MARINDI MPHO	223068894
RAMAKOPA KOKETSO	224060934
ZULU BONGA	224043843
ZWANE BANELE	224006427
NGCOBO SIBONELO	224007550

1.Design goals, constraints and architecture justification

1.1. Design goals

The system's architectural choices are all driven by design goals, which were formulated directly from the project's objectives and its functional and non-functional requirements.

Design goal	Rationale
Usability	The people using this system volunteers and organisation admins are only expected to know the basics of using computers. That means the interface has to be simple and clear enough for them to use right away, with little to no training.
Security & privacy	The platform stores sensitive personal information contact details, availability, interests, and optional criminal record disclosures, the passwords must be hashed, connections encrypted, access limited by role, and data protection law followed
Reliability	The platform should ensure that applications and participation records are securely stored and that opportunities remain accessible, providing volunteers with reliable and timely information.
Scalability	The platform should be scalable, allowing it to support increasing numbers of volunteers, organisations, and opportunities without requiring major redesigns.
Maintainability	The system should use a modular design, allowing individual components, such as the chatbot's knowledge base, to be updated without rebuilding the entire platform.
Accessibility and cross device support	Since users may access the system from either a desktop or a mobile device, the interface must be responsive and never rely on colour alone to convey meaning.
Trust	Verifying organisations and giving users a way to report suspicious activity are both needed to build confidence that the organisations on the platform are legitimate.

1.2. Constrains

Constraint type	
Technical	The system must be developed as a Progressive Web Application (PWA) for both desktop and mobile use, with PostgreSQL as the relational database. Third party API dependencies may increase integration complexity, while older devices and users with limited connectivity may face support limitations.

Economic	Estimated local development cost of R175,000–R340,000 and annual operating cost of R65,600–R130,400 limit how much can be built before prioritising core functionality.
Schedule	The system is built iteratively within an academic development period, one week building on the last, which limits the scope that can be delivered before assessment deadlines.
Team experience	The team consists of beginner developers, so the architecture must favour well understood, mainstream technologies over experimental ones.
Scope control	Requirements were prioritised using MoSCoW, new features must be evaluated before being added, since uncontrolled scope growth is a documented risk (R7).
Legal/ Compliance	The system must follow data-protection regulation (POPIA) given the personal data it handles, including a volunteer's optional criminal record disclosure.

1.3. Architecture Justification

Based on the stated goals and constraints, a layered three-tier client–server architecture was selected. It consists of a Progressive Web Application (PWA) presentation layer, an API-based application and business-logic layer, and a PostgreSQL database layer.

- Progressive Web Application: A PWA provides access on both desktop and mobile devices through a single codebase, reducing development cost and time while supporting the project's accessibility requirements.
- Relational Database (PostgreSQL): PostgreSQL is suitable for storing structured and related data such as volunteer profiles, organisations, opportunities, applications, and attendance records. It maintains data integrity and supports efficient searching, filtering, and matching.
- Modular Components: Separating functions such as authentication, matching, chatbot, notifications, attendance, and administration improves maintainability. Individual components can be updated independently without rebuilding the entire system.
- Role Based Access Control: Access control ensures that volunteers, organisation administrators, and system administrators can only access the features and information relevant to their roles, improving system security.
- Stateless API: A stateless API allows additional server instances to be added as the platform grows, supporting scalability without requiring major changes to the client.
- Email and Notifications: Automated notifications are used instead of in-app messaging to reduce development complexity while ensuring volunteers receive important reminders and schedule updates on time.

2. System Architecture and Component Design

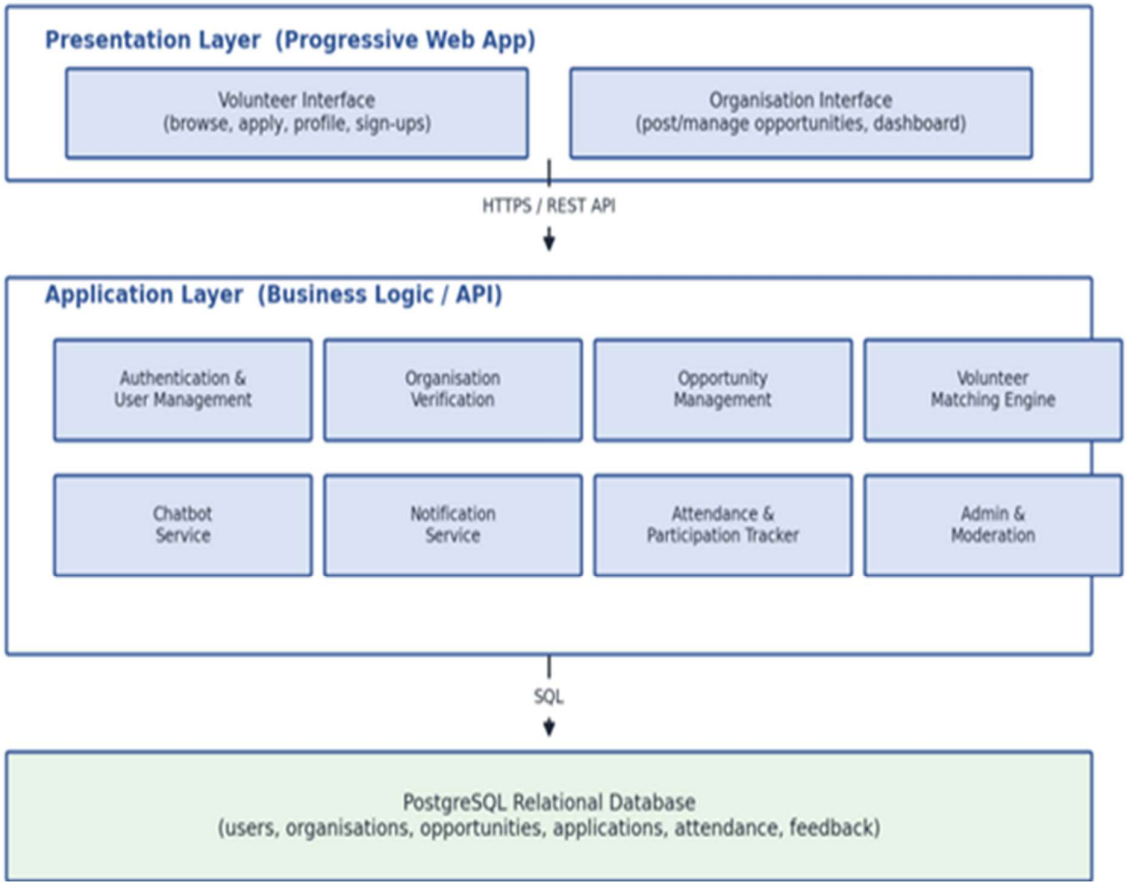
2.1. High Level architecture

The system uses a three-tier architecture consisting of:

- Presentation Layer: A Progressive Web Application (PWA) that provides interfaces for volunteers and organisation administrators, including registration, login, dashboards, opportunity browsing, and profile management.
- Application (Business Logic) Layer: Independent components accessed through an API that handles the system's main functions and business rules.
- Data Layer: A PostgreSQL relational database that stores user accounts, organisations, opportunities, applications, attendance records, and feedback.

This separation means the client only ever talks to the application layer through the API, and the application layer is the only part of the system permitted to talk to the database, which keeps each layer independently replaceable and testable.

Figure 1: Three Tier System Architecture

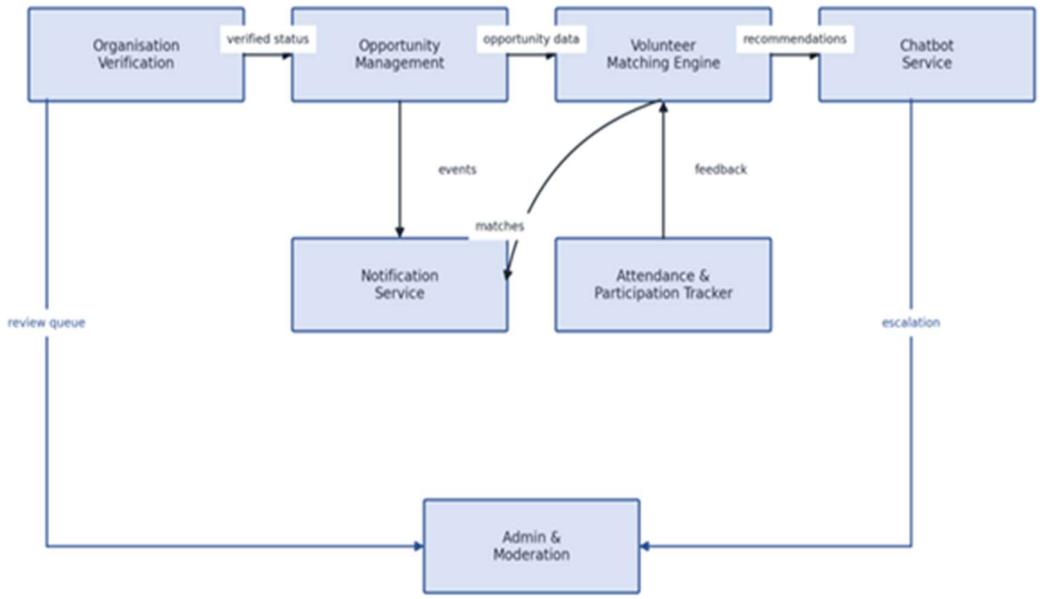


2.2. Component Design

The application layer is divided into the following components, each mapped to the functional requirements it fulfils.

Component	Responsibility	Key Iterations
Authentication & User Management	Handles registration, login, logout, password security, role based access and user profiles.	Verifies user identity and roles for other components.
Organisation Verification	Registers organisations and allows administrators to review and verify organisation details.	Provides verification status to Opportunity Management and Admin & Moderation.
Opportunity Management	Allows organisations to create, edit, publish, and cancel opportunities. Volunteers can browse, search, and filter opportunities.	Communicates with the database, Matching Engine and Notification Service.
Volunteer Matching Engine	Matches volunteers with opportunities based on skills, interests, availability and location.	Uses volunteer and opportunity data and provides recommendations to the Chatbot and Notification Service.
Chatbot Service	Answers common questions, provides guidance, recommends opportunities, and escalates complex queries.	Uses opportunity and matching data. It's knowledge base can be updated independently.
Notification Service	Sends notifications about applications, acceptances, rejections, reminders and opportunity changes.	Receives events from Opportunity Management, Applications, and Matching Engine.
Attendance & Participation Tracker	Records attendance, completed activities, participation history and post activity feedback.	Updates volunteer records and provide feedback data to the Matching Engine.
Admin & Moderation Module	Manages registered users, reviews reported users, content, removes inappropriate opportunities and monitors overall system activity.	Interacts with other components for system wide oversight and receives escalations.

Figure 2: Component iteration diagram

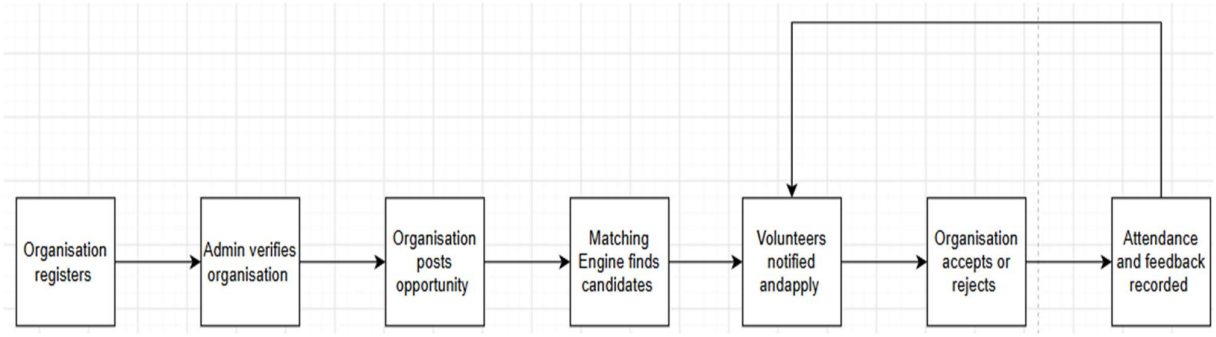


2.3. Data Flow Overview

The typical interaction between the system components follows this process:

- Organisation Registration: An organisation registers and submits its details for verification. The Organisation Verification component sends the submission to an administrator for approval.
- Opportunity Creation: Once verified, the organisation uses Opportunity Management to create and publish an opportunity containing details such as required skills, location, date, and available spaces.
- Volunteer Matching: The Matching Engine compares the opportunity with volunteer profiles and generates suitable recommendations. The Notification Service alerts relevant volunteers.
- Application: A volunteer searches for or accepts a recommendation and submits an application. The application is recorded, available spaces are updated, and the organisation is notified.
- Application Outcome: The organisation accepts or rejects the application, and the Notification Service informs the volunteer of the outcome.
- Participation: After the activity, the Attendance & Participation Tracker records attendance and collects feedback. This information can be used by the Matching Engine to improve future recommendations.
- Chatbot and Administration: Throughout the process, the Chatbot Service assists users with questions and application guidance. Unresolved issues are escalated to Admin & Moderation, which also monitors reported users and content.

Figure 3: Opportunity Lifecycle data flow

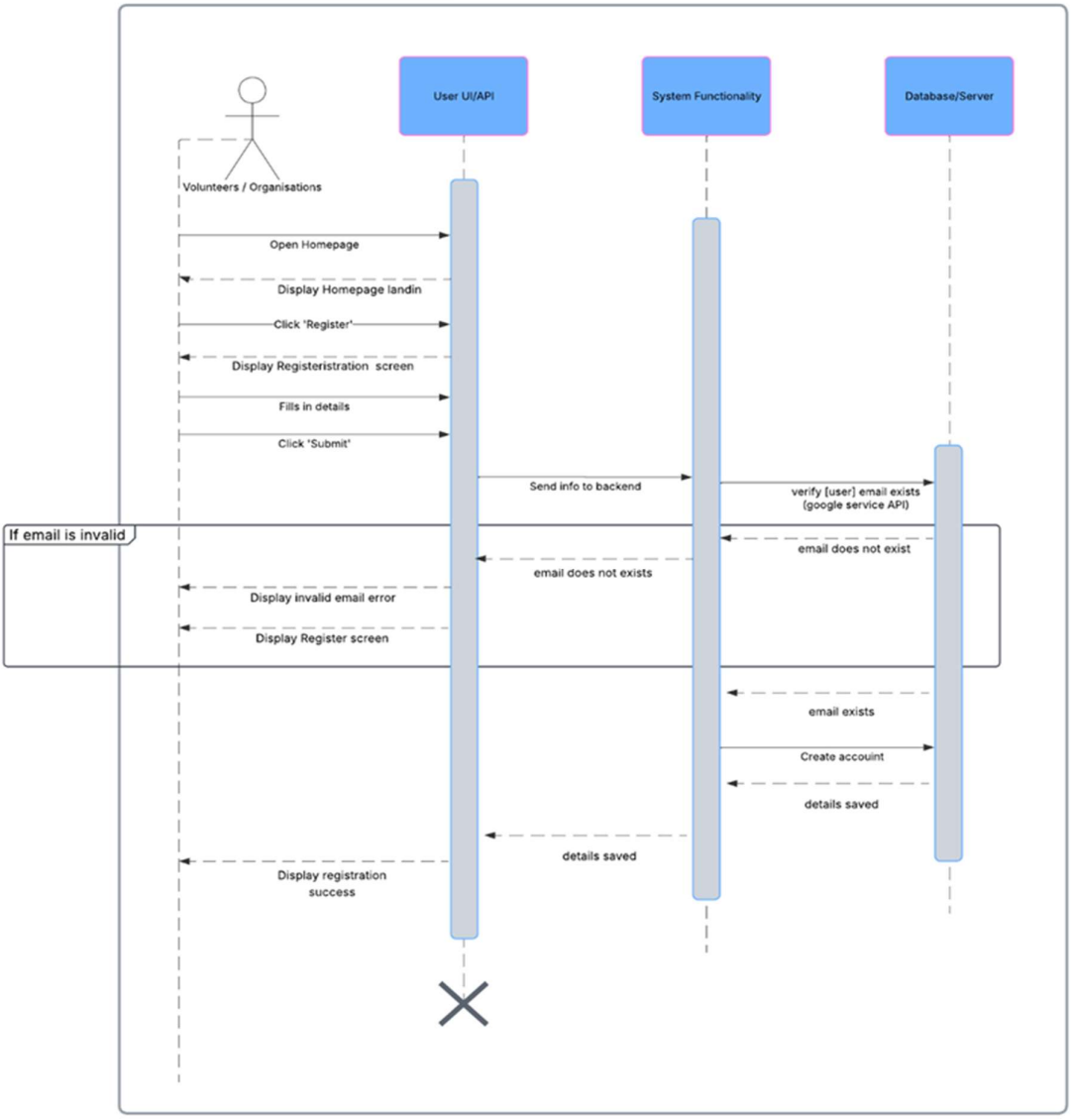


3. Design Diagrams

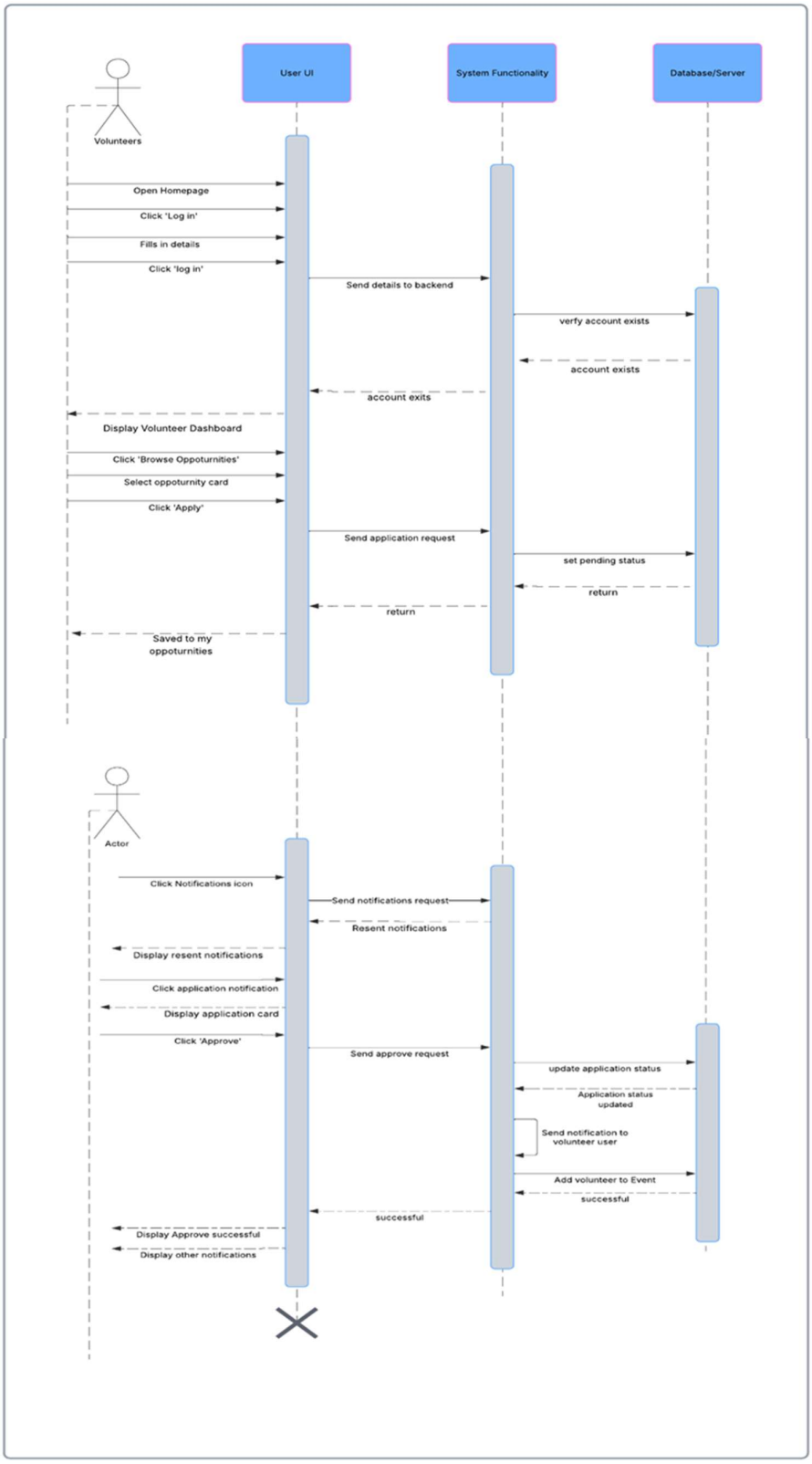
Sequence Diagrams

For commands, the User will interact with the System GUI which will invoke the system's functionality to execute specified functions or methods. These functions or methods will interact with the Server/Database to verify, append or extract information needed for them to be executed. The commands and responses will be sequential.

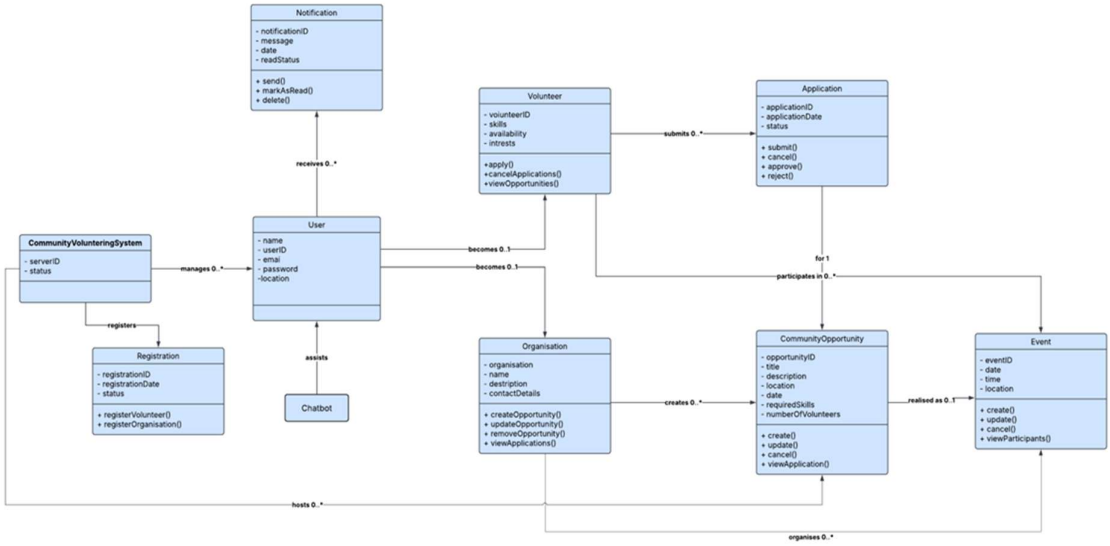
- **Registration**
  - **Case**: A new user wants to register into the system.



- **Applications**
  - **Case**: A volunteer applies for a volunteering opportunity and an organization admin approves it (after a proper review).

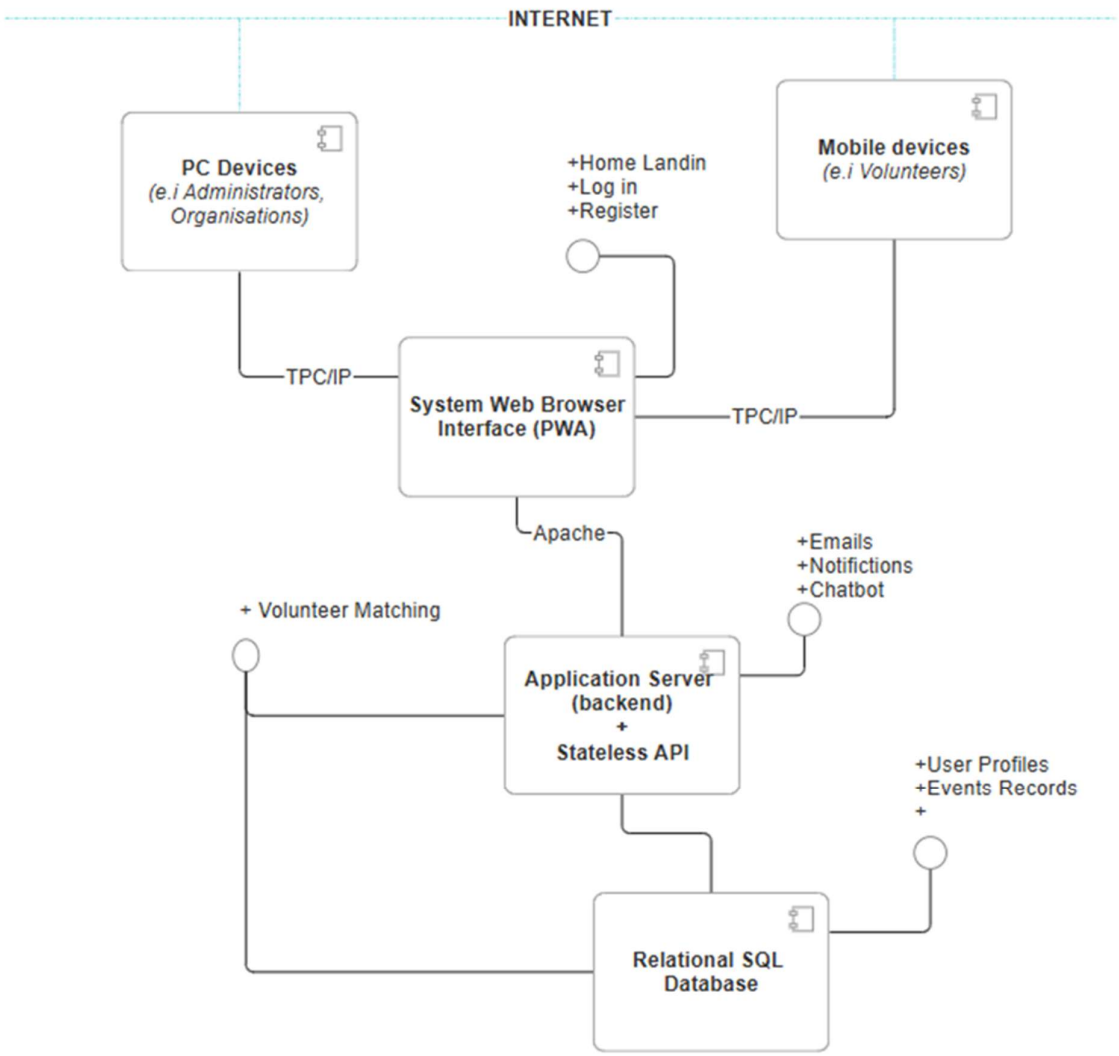

User UI
System Functionality
Database/Server
Volunteers
Open Homepage
Click 'Log in'
Fills in details
Click 'log in'
Send details to backend
verfy account exists
account exists
account exits
Display Volunteer Dashboard
Click 'Browse Oppoturnities'
Select oppoturnity card
Click 'Apply'
Send application request
set pending status
return
return
Saved to my oppoturnities
Actor
Click Notifications icon
Send notifications request
Resent notifications
Display resent notifications
Click application notification
Display application card
Click 'Approve'
Send approve request
update application status
Application status updated
Send notification to volunteer user
Add volunteer to Event
successful
successful
Display Approve successful
Display other notifications

Class Diagram



- **Community Volunteering System**: This is the main class that over sees the system and will provide a report of the functioning status of the whole system.
- **Register**: This class is responsible for the user registration process which will be overseen by the system admin.
- **User**: This is the user base class from which the **Volunteer** and **Organization** classes will inherit basic information attributes like name and userID from.
- **Volunteer**: This class represents the volunteers which will be using this system. They will be able to apply for and view volunteering opportunities. They will also be able to receive and manage notifications about volunteering opportunities.
- **Organization**: This class represents the organisations(admin) that will be using this system. They will be able to create and manage volunteering opportunities and view volunteer applications.
- **VolunteeringOpportunity**: This class represents volunteering opportunities which will be managed by organisations and viewed by volunteers via notifications and posts.
- **Event**: This class represents the volunteering activities that will take place.
- **Notifications**: This class will handle the notifications received by the volunteer.
- **Chatbot**: The chatbot will assist the users with queries and act as search information regarding the system.

Deployment Diagram



- **PC and Mobile devices** are physical devices that will be able to able to access the system through web browsers.
- **System Web Interface (PWA)** will receive TPC/IP requests from physical devices. The requests will be managed by a handler software (e.i Apache) which will then communicate with the server.
- **Application Server and Stateless API** will handle the business logic of the system and cater for end-user functionality.
- **Relational SQL Database** will keep and link user data and volunteering event records. It will also follow standard privacy and security protocols.

4. Database Design Overview

The database is part of the system that remembers all important information. It stores users, organisations, volunteer opportunities, applications, skills, attendance, and notifications. A relational database such as PostgreSQL is appropriate because these pieces of information are connected to one another.

How the design was produced

The simplest design method is read the requirements, identify the important nouns, turn those nouns into tables, give every table a primary key, and connect related tables with foreign keys

Requirement idea	**Database table(s)**
Users register and log in	USERS, VOLUNTEER PROFILE, ORGANISATION, ORGANISATION MEMBER
Organisations post opportunities	ORGANISATION, OPPORTUNITY
Volunteers apply	VOLUNTEER PROFILE, APPLICATION, OPPORTUNITY
Match by skills and interests	SKILL, VOLUNTEER SKILL, OPPORTUNITY SKILL, INTEREST, VOLUNTEER INTEREST
Track participation	ATTENDANCE
Send reminders / decision	NOTIFICATION
Post-activity feedback	FEEDBACK
Chatbot knowledge	CHATBOT FAQ

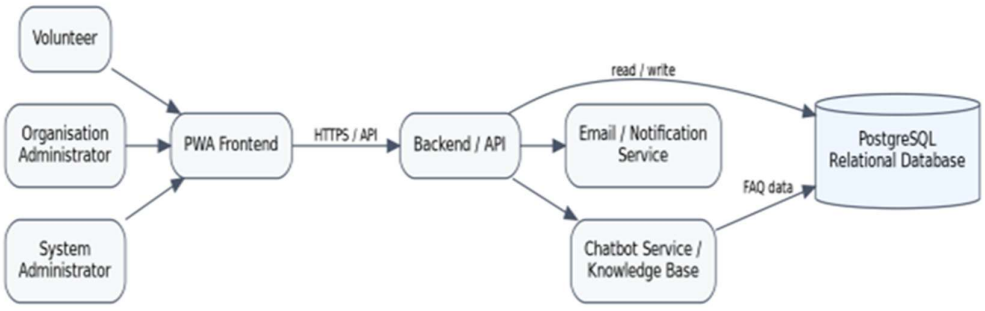


Figure 1. Database context inside the complete platform.

A volunteer is connected to an opportunity through an application. The status is changed, and a notification can be made when the organization accepts or rejects it. In the event that a volunteer is accepted, attendance documents their full participation.

Main Tables

An entity is a major thing that the system needs to store. Each entity normally becomes a table.

Table	Purpose	Example data
USERS	Common login/account information	email, password hash, role
VOLUNTEER PROFILE	Volunteer personal and availability information.	name, phone, location
ORGANISATION	Community groups that need volunteers.	name, address, verification
ORGANISATION MEMBER	Links organisation administrators to organisations.	User ID and Organisation ID
OPPORTUNITY	Volunteer activities posted by organisations.	title, date, location, spaces
APPLICATION	Volunteer request to join an opportunity.	volunteer, opportunity, status
ATTENDANCE	Completed participation and hours.	Check in, check out, hours
SKILL / INTEREST	Information used for matching	First Aid, Teaching, Environment
NOTIFICATION	Application decisions and reminders	message, type, read status
FEEDBACK	Post-activity comments.	rating, comments
CHATBOT FAQ	Editable chatbot knowledge.	question, answer

Entity Relationship Diagram (ERD)

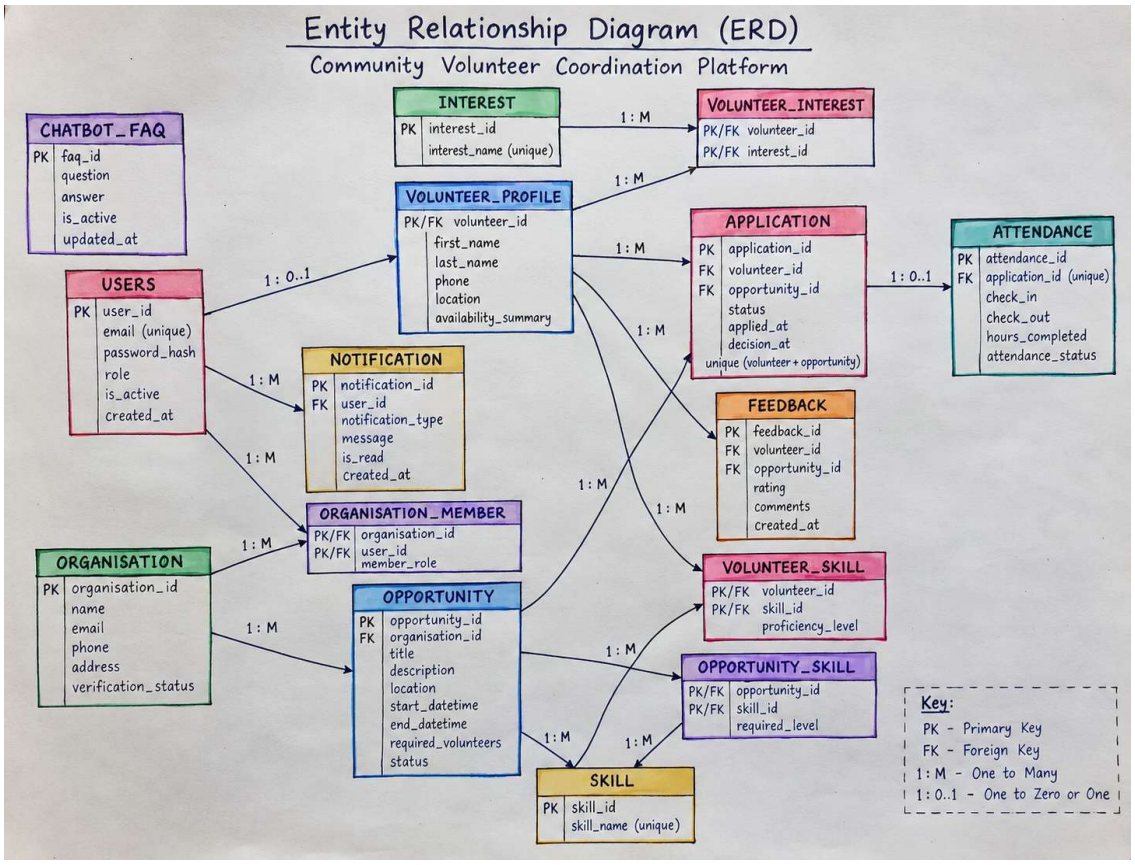


Figure 2. Proposed relational ER diagram

4.2 Core Table Definitions

Users

Field	Type	Key/rule	Meaning
User ID	BIGSERIAL	PK	Unique user
Email	VARCHAR (255)	UNIQUE, NOT NULL	Login email
Password Hash	VARCHAR (255)	NOT NULL	Secure password hash
role	VARCHAR (30)	NOT NULL	VOLUNTEER / ORG ADMIN / SYSTEM ADMIN
Is Active	BOOLEAN	DEFAULT TRUE	Account state

Created At	TIMESTAMP	DEFAULT CURRENT TIMESTAMP	Creation time

VOLUNTEER_PROFILE

Field	Type	Key/rule	Meaning
Volunteer ID	BIGINT	PK + FK USERS	Volunteer account ID
First Name	VARCHAR (100)	NOT NULL	First name
Last Name	VARCHAR (100)	NOT NULL	Surname
Phone	VARCHAR (30)		Contact number
Location	VARCHAR (150)		Used for matching/search
Availability Summary	VARCHAR (255)		Volunteer availability

ORGANISATION

Field	Type	Key/rule	Meaning
Organisation ID	BIGSERIAL	PK	Unique organisation
name	VARCHAR (200)	NOT NULL	Organisation name
email	VARCHAR (255)		Contact email
phone	VARCHAR (30)		Contact number
address	VARCHAR (255)		Location/address
Verification Status	VARCHAR (30)	NOT NULL	Pending / Verified / Rejected

OPPORTUNITY

Field	Type	Key/rule	Meaning
Opportunity ID	BIGSERIAL	PK	Unique opportunity
Organisation ID	BIGINT	FK ORGANISATION	Posting organisation
title	VARCHAR (200)	NOT NULL	Activity title
description	TEXT	NOT NULL	Full details
location	VARCHAR (200)	NOT NULL	Activity location
Start Date and Time	TIMESTAMP	NOT NULL	Start
End Date and Time	TIMESTAMP		End
Required Volunteers	INTEGER	CHECK >= 0	Capacity
status	VARCHAR (30)	NOT NULL	Open / Closed / Cancelled / Completed

APPLICATION

Field	Type	Key/rule	Meaning
Application ID	BIGSERIAL	PK	Unique application
Volunteer ID	BIGINT	FK VOLUNTEER_PROFILE	Applicant
Opportunity ID	BIGINT	FK OPPORTUNITY	Target activity
status	VARCHAR (30)	NOT NULL	Pending / Accepted / Rejected / Withdrawn
Applied At	TIMESTAMP	DEFAULT CURRENT_TIMESTAMP	Submission time
Decision At	TIMESTAMP		Decision time

Supporting Tables and Application Flow

Table	Main keys	Purpose
ORGANISATION MEMBER	User ID and Organisation ID	
SKILL	Skill ID	Master list of skills
VOLUNTEER SKILL	Volunteer ID and Skill ID	Connects volunteers to skills
OPPORTUNITY SKILL	Opportunity ID and Skill ID	Stores skills requested by opportunities
INTEREST	Interest ID	Master list of interests
VOLUNTEER INTEREST	Volunteer ID and Interest ID	Connects volunteers to interests
ATTENDANCE	Attendance ID and Application ID	Records attendance and completed hours
NOTIFICATION	Notification ID and User ID	Stores reminders and status updates
FEEDBACK	Feedback ID	Stores optional post-activity feedback
CHATBOT FAQ	FAQ ID	Stores editable chatbot Q&A items

Example data flow

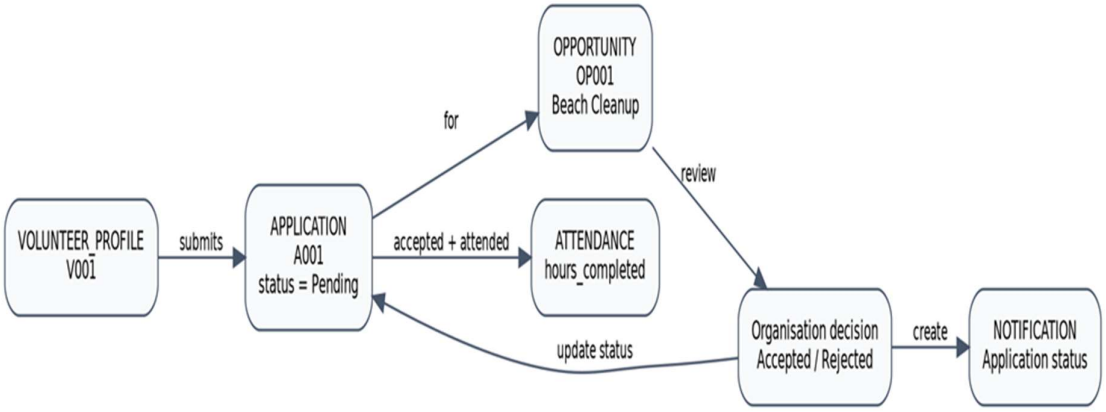


Figure 3. Application decision and attendance flow.

A volunteer is linked to a volunteer opportunity through an application. The volunteer might receive a notification, and the application status is updated when the organization accepts or rejects their application. The volunteer's involvement and completed hours are noted in Attendance if they are accepted and participate in the activity

Relationship and Integrity Rules

Rule	**Meaning**
ORGANISATION 1: M OPPORTUNITY	One organisation can create many opportunities
VOLUNTEER 1: M APPLICATION	One volunteer can submit many applications
OPPORTUNITY 1: M APPLICATION	One opportunity can receive many applications
UNIQUE (volunteer id, opportunity id)	Prevents a volunteer from applying twice to the same opportunity
APPLICATION 1: 0.1 ATTENDANCE	Attendance is only recorded when participation occurs
VOLUNTEER M: N SKILL	Implemented using VOLUNTEER SKILL
OPPORTUNITY M: N SKILL	Implemented using OPPORTUNITY SKILL
VOLUNTEER M: N INTEREST	Implemented using VOLUNTEER INTEREST

Normalization

Normalization helps to keep the database organized and reduce repeated information. For example, skills are stored once in the Skill table and then linked to volunteers using the Volunteer Skill table. This avoids entering the same information many times and makes the database easier to manage.

Security

Security protects users and their personal information. Passwords should be stored securely using hashing instead of plain text. Users should also only be allowed to access functions related to their roles. Important information such as applications and attendance records should be stored safely for future reference

Performance

Information that users frequently look for, such email addresses, opportunity places and times, volunteer applications, and notifications, should be indexed. This facilitates quicker information retrieval in the database.

5. User interface Design and wire frames

5.1 User-Interface Design

The user-interface design of the community volunteering platform will be designed to provide a simple and user-friendly system for the two main users: Volunteers and Organization Administrators. The interface will be designed to be used on both desktop and mobile devices because the system will operate as a Progressive Web Application (PWA).

The design will focus on making the most important functions easy to access without requiring any training. These functions include viewing opportunities, searching for opportunities, applying for opportunities, managing profiles, posting opportunities, and managing volunteers.

5.2 Volunteer Interface

The volunteer interface will provide access to the following features:

- Dashboard
- Browse opportunities
- Search opportunities
- View opportunity details
- Apply for opportunities
- View applications
- View upcoming activities
- View participation history
- Manage personal profile
- Access the chatbot
- View notifications

5.3 Administrator Interface

The administrator interface will have the following features:

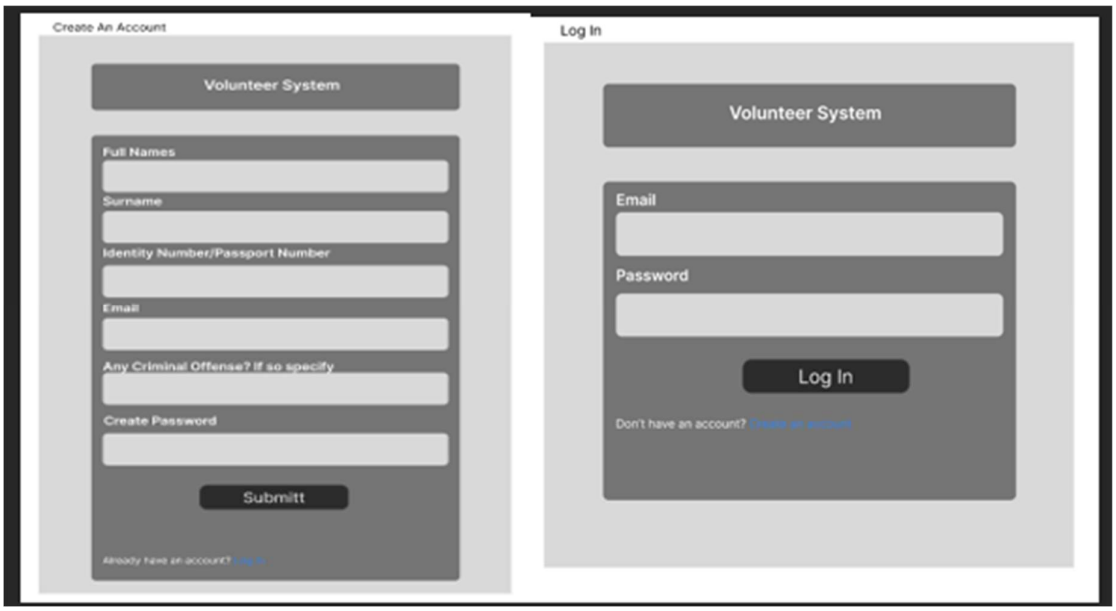
- Dashboard
- Post opportunities
- View posted opportunities
- Edit the posted opportunities
- Manage the applications
- Assign the volunteers
- Manage the organization's profile
- View volunteer participation

5.4 Wireframes

Wireframes will be created for the following:

5.4.1 Volunteer Wireframe

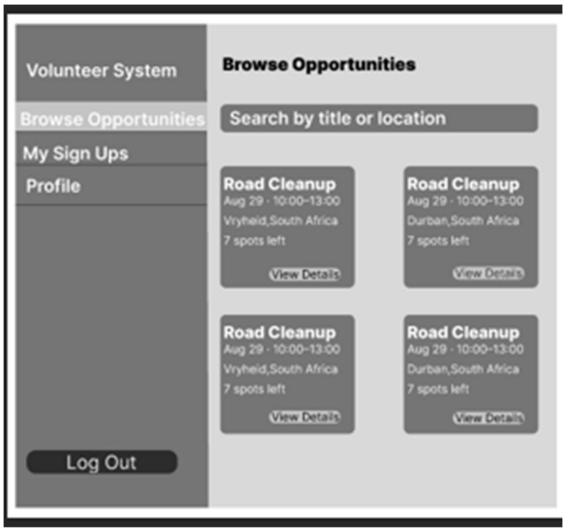
Wireframe 1 – Login/Register



Displays:

- Email
- Password
- Log in button
- Register button
- Forgot password

Wireframe 2– Browse Opportunities



Displays:

- Search bar
- Filters
- Opportunity cards
- Date
- Location
- Available spaces
- "View Details" button

Wireframe 3– Opportunity Details



Displays:

- Opportunity name
- Description
- Date and time
- Location
- Required skills
- Number of available spaces
- Apply button

Wireframe 4– Volunteer Profile

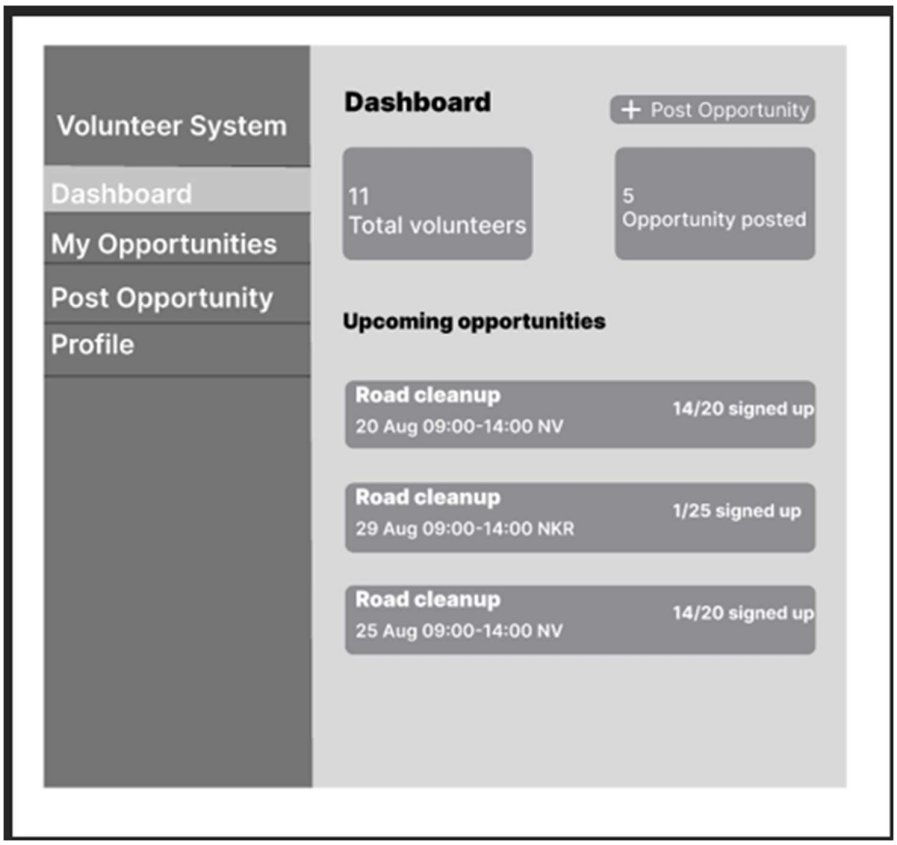


Displays:

- Personal information
- Skills
- Interests
- Availability
- Location
- Edit profile button

5.4.2 Administrator Wireframes

Wireframe 5– Organization Dashboard



Displays:

- Active opportunities
- Applications
- Assigned volunteers
- Upcoming activities
- Navigation menu

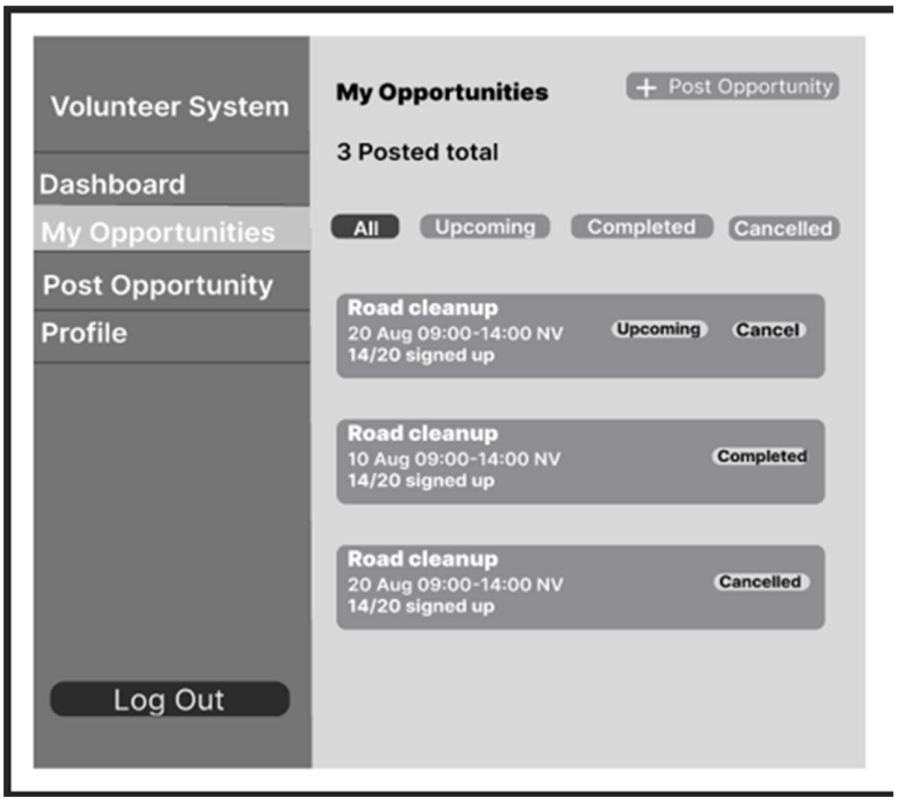
Wireframe 6– Post Opportunity



Displays:

- Opportunity name
- Description
- Location
- Date
- Time
- Required skills
- Number of volunteers
- Submit/Post button

Wireframe 7– My Opportunities



Displays:

- Posted opportunities
- Number of applicants
- Available spaces
- Edit button
- Delete button
- View application button

6. API/Interface or Hardware Design

6.1 API and Interface Design

The Community Volunteer Coordination System will be developed as a Progressive Web Application (PWA), which will allow volunteers and organizational administrators to be able to access the system on mobile devices and desktops. The interfaces will provide access to different functions depending on the user's needs.

The interface will have to main interfaces:

- **The volunteer interfaces:** Allows the volunteer to login, manage a profile, browse and search opportunities, apply for activities, view their applications and participation history.
- **The Organization Administrator Interface:** Allow administrators to register the organization, manage their organizations profile, create and manage volunteer opportunities, and monitor volunteer applications.

The interface will communicate with the system backend through API. The API will be responsible for communication between user interface, application logic, and database.

API Operation	Function
User Registration	Create a new volunteer or organizational account
User Login	Verify users and identifies roles
Profile Management	Creates and updates volunteers and organizational information
View Opportunities	Retrieve available volunteer information
Search and browse opportunities	Finds opportunities based on the volunteer's search
Create Opportunities	Allow organizations to publish new activities
Apply for Opportunity	Allow volunteers to apply for activities
Manage Applications	Allow organizations to accept and rejects application
Notifications	Provide the users with application and opportunity updates
Attendance/Participation	Records and be able to retrieve volunteer participation information

The API will be role-based, which will allow users to only access appropriate functions for their role. For example, a volunteer should not be able to access organizational information.

The system will support automated notifications and email.

6.2 Hardware Requirements

No specialized hardware is required for the Community Volunteer Coordination System. The system is designed to use devices that have access to the internet.

The expected hardware includes:

- Desktop
- Smart devices
- A server or cloud hosting environment for running the application.
- Database storage for storing system information.

6.3 Interface Design Considerations

The interface will be developed according to the usability and accessibility requirements defined in the first submission. The interface should be simple, intuitive, and user-friendly so that users with basic computer skills can understand the main functions without extended training. If an operation fails, clear error messages will also be provided.

The interface will also:

- Use clear and legible text.
- Offer consistent navigation.
- Use clear labels on buttons and forms;
- Don't use color as the only way to convey information.
- Responsive layout for desktop and mobile devices.
- Make the most common functions easy to find.
- Provide understandable feedback on whether an action was successful or not.

These design decisions help to support the project's accessibility requirements and mitigate the risk of bad user adoption due to a difficult interface.

7.Coding standards and development conventions

7.1 Naming Conversion

Meaningful and descriptive names will be used for classes, methods, and attributes so that their purpose can be easily understood by developers working on the system.

- **Classes**

Classes will use PascalCase, where each word starts with a capital letter. Each class represents a major entity or responsibility within the system

Class	**Purpose**

User	Represents common information and functionality shared by users of the system.
Volunteer	Represents a volunteer registered on the system and stores information related to their account and volunteering activities.
OrganisationAdmin	Represents an administrator responsible for managing an organisation and its volunteering opportunities.
Organisation	Represents a community organisation that provides volunteering opportunities.
Opportunity	Represents a volunteering opportunity posted by an organisation, including its description, location, date, time and capacity.
Application	Represents a volunteer's application or registration for a particular volunteering opportunity.
Notification	Represents notifications sent to users about relevant system activities, such as applications or opportunity updates.

- **Methods**

Methods will use camelCase and their names will describe the action that the method performs.

Method	**Purpose**
registerUser()	Creates a new user account in the system.
loginUser()	Authenticates a user using their login credentials.
viewOpportunities()	Retrieves and displays available volunteering opportunities.
searchOpportunities()	Searches for opportunities based on specified criteria.
viewOpportunityDetails()	Retrieves the detailed information of a selected opportunity.
applyForOpportunity()	Records a volunteer's application for an opportunity.

createOpportunity()	Creates a new volunteering opportunity for an organisation.
updateOpportunity()	Updates information associated with an existing opportunity.
viewVolunteers()	Retrieves volunteers who have registered for an opportunity.
updateProfile()	Updates the user's profile information.
changePassword()	Allows a user to change their account password.

- **Attributes**

Attributes will use camelCase and describe the data stored by an object. The names should clearly indicate what information the attribute represents.

Attribute	**Purpose**
userId	Uniquely identifies a user.
firstName	Stores the user's first name.
lastName	Stores the user's surname.
email	Stores the user's email address.
password	Represents the user's account password or password credential.
organisationId	Identifies the organisation associated with an administrator or opportunity.
organisationName	Stores the name of an organisation.
opportunityId	Uniquely identifies a volunteering opportunity.
title	Stores the name/title of a volunteering opportunity.
description	Stores a description of the volunteering activity.
location	Stores where the volunteering activity will take place.

opportunityDate	Stores the date on which the activity takes place.
startTime	Stores the starting time of the activity.
capacity	Stores the maximum number of volunteers allowed for an opportunity.
volunteerCount	Stores or represents the number of volunteers currently registered for an opportunity.
applicationId	Uniquely identifies a volunteer application.
applicationStatus	Represents the current status of an application.

7.2 Code Structure and Organisation

The system will be organised into logical components based on their responsibilities. User-interface code, business logic, database operations and API functionality should be separated where possible. This reduces unnecessary dependencies between components and makes individual sections of the system easier to modify and test.

7.3 Code Style, Formatting and Tooling

- ESLint with a single shared team config enforces syntax rules (no unused variables, consistent equality checks, no implicit globals). No individual override is committed.
- Prettier owns all formatting indentation, quote style, trailing commas, line length. Formatting is never a discussion point in code review because it isn't a human decision.
- Pre-commit hook (Husky + lint-staged): runs lint and format checks automatically before a commit is accepted, so style issues are caught locally rather than in review.
- Editor config: a shared .editorconfig (2-space indentation, UTF-8, LF line endings) keeps behaviour consistent across different editors on the team.

7.4 Error Handling Convention

- Standard server error responses: API errors will use a consistent response structure, such as { error: { code, message } }, rather than exposing raw stack traces or returning an unexplained server error.
- User-friendly client-side errors: Errors that affect users will be displayed through the user interface rather than only being recorded in the browser console. For example, if a volunteer attempts to register for an opportunity that has just reached capacity, the system could display “This opportunity is now full.”

- Server-side error logging: Unexpected errors will be logged on the server with sufficient information to assist with troubleshooting, such as the request path, timestamp and relevant user or transaction identifier. Sensitive information such as passwords will not be included in logs.
- Input validation: User input will be validated before it is processed. For example, an organisation administrator cannot create an opportunity without providing required information such as the title, date, location and volunteer capacity.

7.5 Security Convention

- Password protection: User passwords will never be stored in plain text. Passwords will be securely hashed before being stored in the database and will not be included in application logs.
- Authentication: Functions that modify system data will require the user to be authenticated. For example, only logged-in organisation administrators will be allowed to create or modify their organisation's volunteering opportunities.
- Server-side validation: All input will be validated on the server, even when the same information has already been validated on the client side. Client-side validation improves the user experience but will not be treated as a security boundary.
- Secure error messages: Error messages presented to users will provide enough information to explain what went wrong without exposing sensitive system information, database details or server implementation details.

8. Design Decisions and Trade offs

Design Decision	Advantage	Trade off
SQL relational database	Can handle structured data and relationships between the users and interfaces of this system very well.	Not well suited to frequently changing data structures
Volunteer matching (engine)	Automates linking available volunteering opportunities to volunteers base on their interests and skills (recommendation).	Require more code complexity and detailed user profiles.
Chatbot	Can act as a quick search engine to address queries and provide information	Increase in code complexity and limited responses to queries
Progressive Web App	This allows wide accessibility of the software as it can work remotely from any device with an internet browser. This requires only one base code.	Limited to functions/features that can be released in web format.

Modular components	Splitting the base code into parts improves maintainability of the entire system.	Individual components will require more coherency which can complicate the architecture.
Application layer	Allows communication of multiple interfaces in the backend to supply user with a single simple interface.	This requires more API's to be developed, documented and maintained

9. Requirements-to-Design Traceability

This table proves that the database design directly supports the group's requirements

Requirement	**Database response**
Volunteer/organisation account creation and secure login	USERS plus role specific profile/organisation tables
Profile creation and updating	VOLUNTEER PROFILE, ORGANISATION, ORGANISATION MEMBER
Skills, interests and availability	VOLUNTEER SKILL, SKILL, VOLUNTEER INTEREST, INTER VOLUNTEER PROFILE
Organisation creates opportunities	OPPORTUNITY linked to ORGANISATION
Opportunity location, date, time, required skills and spaces	OPPORTUNITY plus OPPORTUNITY SKILL
Browse/search/filter opportunities	Searchable/indexed OPPORTUNITY fields and skills
Volunteer applies	APPLICATION
Accept/reject applications	APPLICATION status and decision at
Volunteer matching	Volunteer skills/interests/location/availability compared with opportunity data
Notifications and reminders	NOTIFICATION
Attendance and participation history	ATTENDANCE plus historical APPLICATION rows
Post-activity feedback	FEEDBACK
Registered-user administration	USERS role, USERS is active, ORGANISATION verification status
Chatbot support	CHATBOT FAQ

9.1 Recommended Implementation Order

Step	**Action**
1	Create USERS, VOLUNTEER PROFILE, ORGANISATION and ORGANISATION MEMBER
2	Create OPPORTUNITY and link it to ORGANISATION
3	Create APPLICATION and link volunteers to opportunities

4	Add SKILL, INTEREST and their linking tables for matching
5	Add ATTENDANCE and NOTIFICATION
6	Add FEEDBACK and CHATBOT FAQ as supporting features
7	Add foreign key constraints, unique rules and indexes
8	Insert test data and test each requirement using the traceability table

10. Design Review, Consistency and Professional Presentation

10.1 Design Review

The proposed user interface will be reviewed to verify that it meets the required requirements. The review will compare the wireframes against the user's stories and acceptance criteria.

The team will conduct a clear review of the wireframes to identify:

- Missing functionality
- Confusing navigation
- Unlabeled buttons
- Features that do not match with the requirements
- Accessibility problems

10.2 Design Review

Consistency will be maintained throughout all system screens.

The following design conventions will be used:

- The same navigation structure across pages.
- Consistent button names and positions.
- Consistent typography and text sizes.
- Consistent spacing between interface elements.
- Consistent form of layouts.
- Consistent errors and success messages.
- Consistent icons and symbols.
- Similar layouts for related functions.

10.3 Accessibility

The interface will consider accessibility requirements that are required for the system.

The system will:

- Use readable text.
- Provide clear navigation.
- Avoid relying on colour alone to communicate information.
- Use a responsive layout.
- Support both desktop and mobile devices.
- Provide clear and understandable error messages.

10.4 Professional Presentation

The final user interface and the wireframes will be presented in a professional manner.

Each wireframe should have:

- A clear title
- A wireframe number
- Clearly labeled interface components
- Consistent spacing and alignment
- Consistent terminology
- A short description explaining the purpose of the screen

References

[1] PostgreSQL Global Development Group, *PostgreSQL Documentation*. PostgreSQL Documentation

[2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.

[3] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015.

[4] R. S. Pressman and B. R. Maxim, *Software Engineering: A Practitioner's Approach*, 9th ed. McGraw-Hill, 2019.

[5] Lucid Software Inc., *Lucidchart: ER Diagram Tool*. Lucidchart ER Diagram Tool

[6] OWASP Foundation, *Password Storage Cheat Sheet*. OWASP Password Storage Guidance

[7] https://web.dev/explore/progressive-web-apps

[8] https://www.ibm.com/think/topics/api

[9] https://www.coursera.org/articles/ui-design